假面騎士OOO 搜尋引擎 — Final Project Proposal（Group 16）



組員：温亮達、廖若羽、蔡妙青、劉又綺、鄭年芮（Group 16）　程式碼：GitHub repo

1. Introduction

我們要做《假面騎士OOO》專用的搜尋引擎。**無論使用者輸入什麼關鍵字，系統都會找出和「該關鍵字」以及「假面騎士OOO」最相關的內容，並說明為什麼相關。**

例如搜尋「貓咪」時，Google 給的是一般的貓咪資訊；我們的系統會找出 OOO 裡和貓有關的內容：貓系核心硬幣組成的獅虎豹組合、貓系幹部卡佳里、第 3 集與第 13 集的貓怪人，並在每個結果下方附上相關段落、標籤與圖片。下圖是實際執行的結果：



搜尋「貓咪」的實際結果（2026/11/05）：黃色是查詢詞，綠字是段落所在章節，藍色是標籤

為什麼選這個主題

這部作品是我們的童年回憶，拿來做搜尋引擎也剛好會碰到三個有挑戰性的問題：

- **同一個東西有很多名字：** アンク在不同來源寫成安酷、安卡、Ankh。我們的關鍵字表有 28 個概念、約 120 種中日英寫法，必須合併計數。
- **同一個詞會指不同東西：** 「安卡」會誤中土耳其首都「安卡拉」；「貓」會誤中「熊貓」。
- **作品之間會互相客串：** 台版維基的「假面騎士W」條目提到 OOO 18 次，但主題不是 OOO，只數次數分不出這種頁面。

要贏 Google 的是哪一點

我們不跟 Google 比規模，只比「查 OOO 相關資料時，內容最完整的頁面有沒有排在最前面」，並以 NDCG@10 衡量（2.6 節）。我們預期影片、購物頁這類文字少的結果會下降，專門百科會上升；查日常詞（例如「貓咪」）時差距最明顯。Stage 6 再比較加入我們的檢索結果後，LLM 回答冷門細節是否更正確。

2. Search Tricks

一次搜尋的步驟是：查詢加上「假面騎士OOO」找候選網址 → 爬子頁面建成網站樹 → 每頁計分並加總成網站分數 → 從前幾名網站挑出相關段落 → 從段落找擴充詞再搜尋一次。設計的核心是**分數必須同時來自 OOO 與查詢詞**：只講 OOO、或只講查詢詞的頁面都不該排在前面。

2.1 關鍵字權重 w(k)：由維基百科統計

人工決定收哪些概念與寫法，權重由程式計算。直覺是：**在 OOO 條目裡常出現、在其他假面騎士作品條目裡很少出現的詞，權重就高。**

```
spec(k) = f_OOO(k) / ( f_OOO(k) + f_bg(k) )
a(k)    = ( 1 + ln n(k) ) × spec(k) × β_lead(k)
s(k)    = ½ × ( a_ja(k) / max a_ja  +  a_zh(k) / max a_zh )
w(k)    = 1 + 4 × s(k)
```

f_OOO、f_bg 是詞在 OOO 條目、其他 6 部作品條目（平均）中的頻率；n(k) 是在 OOO 條目中的次數；β_lead 在詞出現於導言時為 1.2；a_ja、a_zh 分別用日版、台版維基計算。例如「核心硬幣」：日版出現 115 次、專屬程度 0.99，a_ja = 5.69（÷ 最高分 7.13 = 0.80）；台版 a_zh = 6.20（÷ 6.67 = 0.93），s ≈ 0.86，w ≈ 4.5。權重落在 1 到 5，例如假面騎士OOO 5.0、核心硬幣 4.5、安酷 4.4。我們不用一般的 IDF：其他作品常因客串提到 OOO 的核心詞，IDF 會把「コアメダル」壓到第 109 名。

2.2 單頁分數 S(p) 與加入查詢詞的 S_q(p)

```
S(p)   = Σ[k ∈ K]  w(k) × c(k, p)
S_q(p) = S(p) × ( α + ln(1 + q(p)) )        α = 0.1
```

c(k, p) 是關鍵字 k（含所有寫法）在網頁 p 正文中的次數，q(p) 是查詢詞（含同義寫法）的次數。c 的數法：用 Stack 跳過 nav、footer 等非正文區塊（HW5）；長的寫法優先比對，「假面騎士OOO」不會再算一次「OOO」；「安卡拉」「熊貓」這類排除詞只佔位置、不計次數。

S_q 用**相乘**而不是相加：只講貓、沒提到 OOO 的寵物網站 S = 0，總分為 0；只講 OOO、沒提到貓的頁面只保留 10%（不設 0，爬蟲才會繼續展開它的子頁面）；兩者都有的頁面最高，q 取 ln 避免重複很多次就無限放大。

2.3 網站分數 Site(r)

```
Site(r) = Σ[v ∈ 樹 T_r]  S_q(v) × 0.5^depth(v)
```

從網站首頁 r 用 BFS 爬子頁面建樹（每棵最多 15 頁、2 層），用後序走訪由下往上加總，越深的頁面分數打越多折（HW6 的樹）。例如萌娘百科首頁的單頁分只有中文維基的四分之一，但整站分數只差約 4%，代表它的子頁面涵蓋得更完整。

2.4 關鍵字擴充（Stage 4）

從前 10 名網站的相關段落（2.5 節）中，找出經常和原關鍵字、查詢詞一起出現的詞：

```
score(x) = Σ[原關鍵字 k]  co(k, x) × ln( N / df(x) + 1 )
weight(x) = 1 + 1.5 × score(x) / max score
```

co(k, x) 是 k 與 x 在 4 個詞以內一起出現的次數（共現圖，鄰接表），N、df 以所有爬到的網頁計算，讓到處都有的泛用詞被壓低（TF-IDF 的想法）。用 PriorityQueue 取前 5 個，權重介於 1 到 2.5，低於原關鍵字的最低權重 2.6；再用前 2 個擴充詞搜尋一次並重新排名，最多 2 輪。查「貓咪」時，第 1 輪擴充出「外科、貪吃、進化」（來自第 3、13 集的集數標題），第 2 輪找到「暹羅貓、シャムネコ」（第 13 集的暹羅貓怪人），這些詞都不是人工提供的。

2.5 段落分數 P_q(t)：說明為什麼相關

```
P_q(t) = ( α + ln(1 + q(t)) ) × ( 1 + Σ[k ∈ K] w(k) × c(k, t) )
```

把網頁切成段落（表格列、清單項目、段落），用 Stack 記錄每段所在的章節路徑。段落 t 必須含查詢詞，而且含 OOO 關鍵字或位在 OOO 條目中，才有資格入選；用 PriorityQueue 從整棵網站樹取前 3 段，再以段落中的 OOO 關鍵字當標籤（例如「卡佳里」「獅虎豹組合」），並用標籤查 Kamen Rider Wiki 的圖片。結果就是第 1 節的圖。

2.6 怎麼判定贏過 Google

挑 6 到 10 組查詢，一半是 OOO 專有名詞（核心硬幣、火野映司…），一半是日常詞（貓咪、冰棒、鳥）。把 Google 與我們的前 10 名合併後打亂，兩位組員在不知道來源的情況下各自評分，取平均作為 rel：

rel	意思
3	同時和查詢詞與 OOO 相關，而且內容完整
2	同時相關，但內容較少或只談一部分
1	只和其中一方相關，或幾乎沒有文字（影片、商品頁）
0	無關，或打不開

```
DCG@10  = Σ[i = 1..10]  ( 2^rel_i − 1 ) / log2( i + 1 )
NDCG@10 = DCG@10 / IDCG@10        IDCG：同一批頁面依 rel 理想排序時的 DCG
```

我們的平均 NDCG@10 高於 Google 才算贏。已手動記錄 5 組查詢的 Google 前 20 筆；初步結果中，查「假面騎士OOO」時中華電信 MOD 的影片頁從 Google 第 3 名降到我們的第 14 名，專門百科排在前面。

計畫中的改良： 長頁面目前占便宜，客串頁面靠單一關鍵字也能拿分。我們打算改用對數詞頻與涵蓋度 S′(p) = Cov(p) × Σ w(k)(1 + ln c(k, p))，Cov(p) 是網頁涵蓋到的關鍵字權重比例；用 Stage 1 的 10 個網頁試算，日版維基與萌娘百科的差距由 13.5 倍降為 2.6 倍，客串頁面則被明顯壓低。

3. System Design

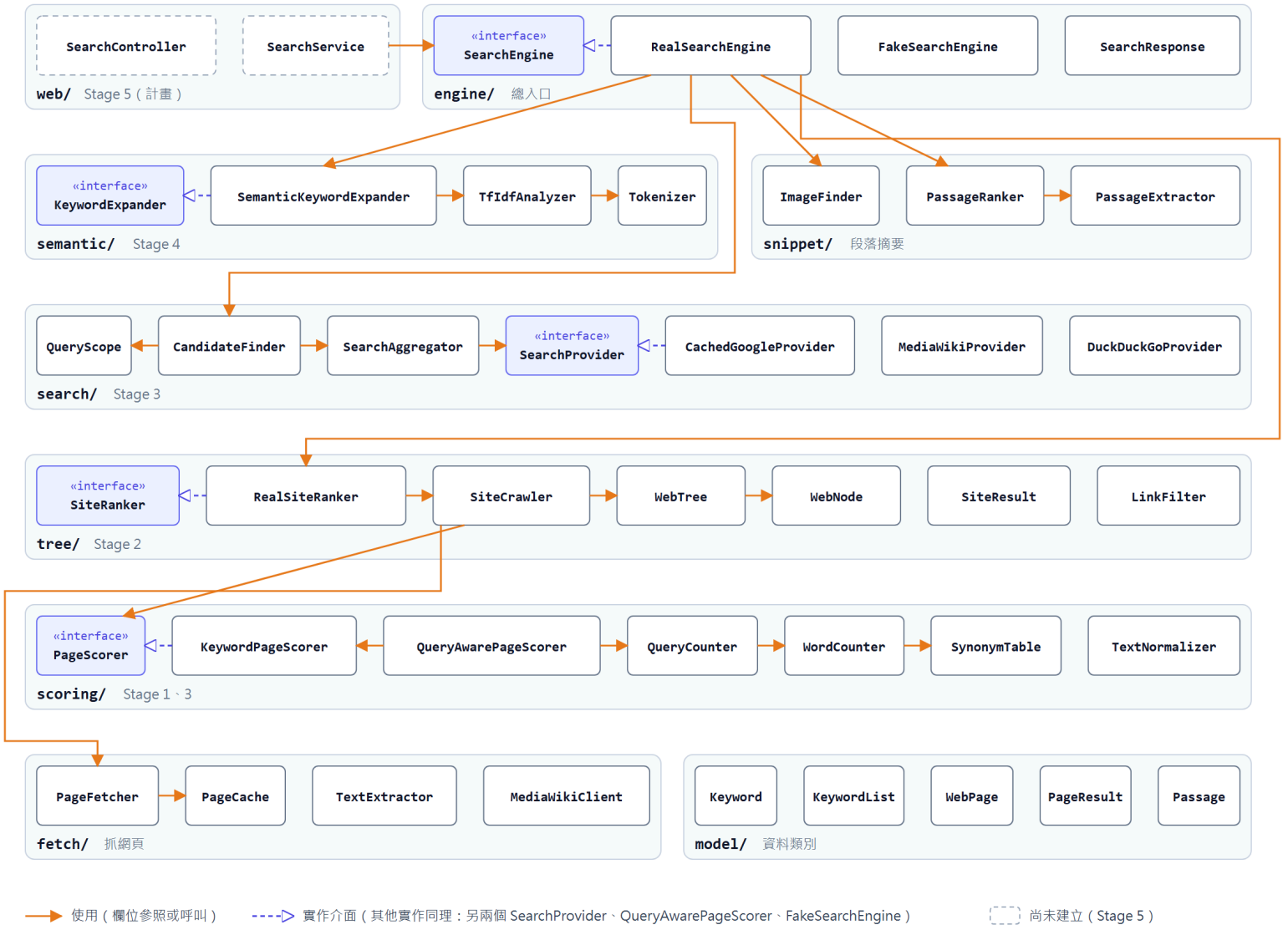
系統是單一 Java Maven 專案，依功能分 package，依賴方向只能由上往下（web → engine → semantic → snippet → search → tree → scoring → fetch → model）。每一層都透過介面對外，各 Stage 可以先用假實作同時開發，寫好再替換；Stage 5 的網站只呼叫 `SearchEngine`。

3.1 一次搜尋的流程


一次搜尋的流程（RealSearchEngine）
Stage 5
使用者輸入任何關鍵字
例如「貓咪」（網站為計畫；目前由命令列 SearchMain 呼叫）
web：SearchController → SearchService → SearchEngine
Stage 3
限定在 OOO 範圍內，找候選網址
查詢加上「假面騎士OOO」與同義詞；Google 快取、維基搜尋、DuckDuckGo 合併去重
search：QueryScope、CandidateFinder、SearchAggregator
Stage 2
爬子頁面建樹
BFS，每棵樹最多 15 頁、2 層；後序走訪加總 Site(r)
tree：SiteCrawler、WebTree、RealSiteRanker
Stage 1、3
每頁計分：OOO 分數 × 查詢詞相關度
S_q(p) = S(p) × (0.1 + ln(1 + q(p)))
scoring：KeywordPageScorer、QueryAwarePageScorer、WordCounter
說明為什麼相關
挑出每個網站的相關段落
Stack 記錄章節路徑，PriorityQueue 取前 3 段，附標籤與圖片
snippet：PassageExtractor、PassageRanker、ImageFinder
Stage 4
從相關段落找擴充詞
jieba 斷詞、共現圖、TF-IDF，PriorityQueue 取前 5 個
semantic：SemanticKeywordExpander、TfIdfAnalyzer、Tokenizer
前 2 個擴充詞加上「假面騎士OOO」再搜尋，以「OOO 關鍵字 + 擴充詞」重新排名，最多 2 輪
Stage 5
回傳結果頁
排名、分數、網站樹、相關段落、標籤、圖片、擴充詞
engine：SearchResponse
Stage 6
離線評估：與 LLM 比較
把相關段落與來源放進 prompt，和直接提問對照（計畫）
llm：PromptBuilder
外框：
擴充迴圈
段落摘要
尚未實作

3.2 Class Diagram

Class Diagram：各 package 的主要類別（依 2026/11/05 的程式碼）



3.3 資料結構對照

資料結構或演算法	用在哪裡	為什麼選它	來源
Stack	TextExtractor、PassageExtractor	跳過巢狀的非正文區塊；記錄段落所在的章節路徑	HW5
字串搜尋 + boolean[]	WordCounter	長的寫法優先，同一位置不重複計數	HW3
樹 + 後序走訪	WebTree	子頁面先算完，再往上加總	HW6
Queue + HashSet（BFS）	SiteCrawler	逐層建樹、不重複爬取，取代手動 addChild	HW6 延伸
介面與多型	SearchProvider、PageScorer 等	新增搜尋來源或計分方式不必改主程式	HW2
HashMap	SynonymTable、PageCache、ImageFinder	寫法對應關鍵字、網址與圖片查詢結果的快取	課堂延伸
LinkedHashSet	CandidateFinder、SearchAggregator	合併多個來源的網址，去除重複且保留順序	課堂延伸
鄰接表 Graph	TfIdfAnalyzer	詞與詞的共現關係	後續課程
PriorityQueue	TfIdfAnalyzer、PassageRanker	取前 K 個擴充詞、前 3 個段落	後續課程

4. Schedule

要交的期限有兩個：11/05 繳 Proposal、12/17 上台 Demo。我們預計 11/26 做完全部 Stage，留 3 週整合、評估與做簡報。



Stage	狀態	負責人	期限	做法
0、1 單頁排名	已完成	温亮達	10/22	介面與假實作；Stack 抽正文、同義詞合併計數、統計權重
2 網站排名	已完成	廖若羽	10/27	BFS 建樹、後序走訪加總
3 自主找網址	已完成	劉又綺、鄭年芮	11/01	多來源搜尋、LinkedHashSet 合併；查詢限定在 OOO、查詢詞加入計分
4 語意擴充	已完成	蔡妙青	11/12	從相關段落斷詞、共現圖、TF-IDF，擴充後再搜尋
正式版搜尋引擎、段落摘要	已完成	温亮達	11/16	串起 Stage 1–4；段落、標籤與圖片
5 上線	未開始	劉又綺、鄭年芮（程式組支援）	11/19	Spring Boot（Controller → Service → SearchEngine）；結果頁顯示排名、網站樹、段落、標籤、圖片；部署到 Render 或 Cloudflare Tunnel；Demo 查詢預先存入快取
6 LLM 比較	未開始	劉又綺、鄭年芮	11/26	10 到 20 題冷門事實題；對照組直接問 ChatGPT、Gemini、DeepSeek，實驗組把相關段落與來源放進 prompt；比較正確性、有無來源、譯名
評估與簡報	未開始	全組	12/10	人工評分、計算 NDCG@10；PPT（負責人待指定）

接下來的重點： Stage 5 網站上線；補記錄日常詞查詢的 Google 結果並完成人工評分；改善擴充詞品質（例如查「核心硬幣」只擴充出泛用詞「聲稱」，改用 2.1 節的專屬程度篩選）；指定 PPT 負責人。

5. Challenges

技術	難在哪裡	學習計畫	備案
擴充詞的品質（Stage 4）	段落中和關鍵字共現的常是句型用詞或泛用詞；jieba 不會切日文	用 2.1 節的專屬程度篩選擴充詞，取代人工排除表	只採用相關度最高的少數擴充詞
上線後的回應時間（Stage 5）	每秒最多抓 1 頁，一次搜尋含擴充要好幾分鐘	學習 Spring Boot 的非同步處理，先回傳第一輪結果再逐步補上	預先爬好 Demo 查詢，線上只讀快取
反爬蟲與動態網頁（Stage 2、3）	百度百科、Facebook 回傳 403 或需要登入；部分網站需要 JavaScript	優先使用 MediaWiki API，尊重速率限制	抓不到的網站標示為「無法評分」
Aho-Corasick（Stage 1 延伸）	要自己實作 Trie 與 failure link，作業只做到單一字串搜尋	先用 Trie 完成比對，再加 failure link，與現有做法比較速度	保留現有的逐一比對，功能不受影響
LLM 評估設計（Stage 6）	同一題每次回答不同，人工評分有主觀偏差	每題固定 prompt、事先寫好標準答案，兩人獨立評分	縮小為 10 題，只比較有標準答案的事實題

希望課堂或助教能協助的有兩項：擴充詞品質的評估方式，以及 Aho-Corasick 的 failure link 建構。